A Mini Project Report on

AI POWERED CODE REVIEWER

Submitted in partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

In

CSE (DATA SCIENCE)

M. Vikas	**(22AG1A6744)**
V. Smaran	**(22AG1A6762)**
E. Augusteen	**(22AG1A6716)**
N. Varun	**(22AG1A6745)**

Under the guidance of

Mr. Shaik Nagur Vali
Assistant Professor

DEPARTMENT OF CSE (DATA SCIENCE)

ACE Engineering College

Ankushapur(V), Ghatkesar(M), Medchal Dist - 501301

(An Autonomous Institution, Affiliated to JNTUH, Hyderabad)

www.aceec.ac.in

A.Y: 2024-2025

ACE
Engineering College
UGC AUTONOMOUS INSTITUTION
(Sponsored by Yadala Satyanarayana Memorial Educational Society, Hyderabad)
Approved by AICTE & Affiliated to JNTUH
B.Tech Courses offered: CIVIL, CSE, IT, ECE, EEE & MECH, NBA Accredited Courses: CIVIL, CSE, ECE, EEE & MECH, Accorded NAAC A - Grade

DEPARTMENT OF CSE (DATA SCIENCE)

CERTIFICATE

This is to certify that the major project report entitled ***"AI Powered Code Reviewer"*** is a Bonafide work done by ***M. Vikas(22AG1A6744), V. Smaran (22AG1A6762), E. Augusteen (22AG1A6716), and N. Varun (22AG1A6745)*** in partial fulfillment for the award of Degree of BACHELOR OF TECHNOLOGY in CSE (Data Science) from JNTUH University, Hyderabad during the academic year 2024 - 2025. This record of Bonafide work carried out by them under our guidance and supervision.

The results embodied in this report have not been submitted by the student to any other University or Institution for the award of any degree or diploma.

Mr. Shaik Nagur Vali Assistant Professor Supervisor	**Dr. P Chiranjeevi** Associate Professor HOD, CSE-DS	**External**

ACKNOWLEDGEMENT

helped me transform an idea into a real-time application. We would like to express my heart-felt gratitude to my parents without whom we would not have been privileged to achieve and fulfill my dreams.

A special thanks to our General Secretary, **Prof. Y. V. Gopala Krishna Murthy,** for having founded such an esteemed institution. Sincere thanks to our Joint Secretary **Mrs. M. Padmavathi**, for support in doing project work. I am also grateful to our beloved **Principal**, for permitting us to carry out this project.

We profoundly thank **Dr. P. Chiranjeevi**, Associate Professor and Head of the Department of Computer Science and Engineering (Data Science), who has been an excellent guide and also a great source of inspiration to my work.
We extremely thank **Mr. G. Parwateeswar and Mrs. B. Saritha,** Assistant Professor, Project coordinator, who helped us in all the way in fulfilling all aspects in completion of our Mini-Project.

We are very thankful to my internal guide **Mr. Shaik Nagur Vali** who has been excellent and also given continuous support for the Completion of my project work. The satisfaction and euphoria that accompany the successful completion of the task would be great, but incomplete without the mention of the people who made it possible, whose constant guidance and encouragement crown all the efforts with success. In this context, we would like to thank all the other staff members, both teaching and non-teaching, who have extended their timely help and eased my task.

M. Vikas	(22AG1A6744)
V. Smaran	(22AG1A6762)
E. Augusteen	(22AG1A6716)
N. Varun	(22AG1A6745)

DECLARATION

We here by declare that the result embodied in this project report entitled **"AI Powered Code Reviewer"** is carried out by us during the year 2024-2025 for the partial fulfilment of the award of **Bachelor of Technology in Computer Science and Engineering (Data Science),** from **ACE ENGINEERING COLLEGE.** We have not submitted this project report to any other Universities/Institute for the award of any degree.

M. Vikas	(22AG1A6744)
V. Smaran	(22AG1A6762)
E. Augusteen	(22AG1A6716)
N. Varun	(22AG1A6745)

AI POWERED CODE REVIEWER

ABSTRACT

An AI-powered code reviewer project is developed using the MERN stack. The application allows users to submit code for review, identifying errors and suggesting improvements. The project includes backend and frontend integration, utilizing AI for code analysis and providing feedback. Utilizing AI in code reviews enhances efficiency, improves code quality, and ultimately leads to better software development practices. Viewers are encouraged to comment for deployment demonstrations. This project overall deals and tackle the challenges like Error Handling, Code Quality and Formatting, CORS Issues, Deployment Challenges, User Interface (UI) Presentation. In modern software development, maintaining high code quality and efficiency is essential. Traditional code review processes can be time-consuming and prone to human error. To address these challenges, this project introduces an AI-powered code reviewer built using the MERN (MongoDB, Express.js, React, Node.js) stack. The system enables users to submit their code for automated analysis, error detection, and improvement suggestions using AI-driven techniques. This project seamlessly integrates backend and frontend technologies, leveraging AI for intelligent code reviews. By automating the review process, the application enhances efficiency, accuracy, and overall software development practices. Furthermore, it tackles key development challenges such as error handling, code quality and formatting, CORS issues, deployment complexities, and UI presentation. By streamlining the code review workflow, this AI-powered system serves as a valuable tool for developers, helping them write better, more maintainable code in real-world applications.

Overall, the project aims users to streamline the code review process, enhance code quality, and address integration issues, making it a valuable tool for developers in real-world scenarios.

CONTENTS

	Chapter Name	PageNo.
CHAPTER - 1	**Introduction**	**1-3**
	1.1 Background and context of the project	
	1.2 Problem Statement and Objectives	
	1.3 Significance and Motivation	
CHAPTER - 2	**Literature Survey**	**4-6**
	2.1 Existing System	
	2.2 Proposed System	
CHAPTER - 3	**Requirement Analysis**	**7-9**
	3.1 Software Requirements	
	3.2 Hardware Requirements	
	3.3 Functional Requirements	
	3.4 Non-Functional Requirements	
	3.5 Environment Setup	
CHAPTER - 4	**System Analysis**	**10-13**
	4.1 Methodology	
	4.2 Modules	
	4.2.1 Data Handling	
	4.2.2 Text Preprocessing & Feature Extraction	
	4.2.3 Model Training & Evaluation	
	4.2.4 Model Prediction	
	4.2.5 Web Application	
	4.2.6 User Management	
	4.2.7 Utility & Support	
CHAPTER - 5	**System Design**	**14-18**
	5.1 System Architecture	
	5.1.1 System Components	
	5.2 UML Diagrams	
	5.2.1 Class Diagram	
	5.2.2 Use Case Diagram	
	5.2.3 Sequence Diagram	
	5.2.4 Activity Diagram	
	5.2.5 Deployment Diagram	

CHAPTER - 6 **Implementation** **19-20**

6.1 Code Structure Overview

CHAPTER - 7 **Output Screens** **23-26**

CHAPTER - 8 **Conclusion, Future Scope and References** **27-29**

LIST OF FIGURES

F.NO	FIGURE NAME	PAGE NO
1	USECASE DIAGRAM	16
2	CLASS DIAGRAM	16
3	SEQUENCE DIAGRAM	16
4	ACTIVITY DIAGRAM	17
5	DEPLOYMENT DIAGRAM	18
6	OUTPUT SCREENS	
	FIG – 6.1	24
	FIG – 6.2	25
	FIG – 6.3	26

INTRODUCTION

INTRODUCTION

1.1 Background and Context of the Project:

In the era of rapid software development, maintaining clean, efficient, and secure code has become increasingly important. Traditional code reviews, which involve peer-to-peer manual inspection, are often time-consuming, error-prone, and inconsistent due to human subjectivity. These limitations create a gap in ensuring code quality, particularly in large-scale projects or fast-paced environments. With advancements in Artificial Intelligence (AI) and Natural Language Processing (NLP), AI-powered tools now have the capability to assist in reviewing source code more accurately and efficiently. Leveraging these technologies, this project introduces an AI-Powered Code Reviewer that automates the code review process using Gemini AI integrated with the MERN (MongoDB, Express.js, React.js, Node.js) stack. This tool is designed to identify common bugs, suggest code improvements, and flag vulnerabilities, thereby streamlining development workflows and improving software quality.

1.1.1 Applications of AI-Powered Code Review:

- Software Development: Automates error detection and enforces coding standards in real-time.
- Open-Source Projects: Ensures consistency across diverse code contributions.
- Education: Helps students receive instant feedback on code quality.
- Enterprise Solutions: Accelerates DevOps and continuous integration (CI) pipelines.

1.1.2 Importance of Automated Code Review in Various Fields:

Automated code review is critical across industries that rely on reliable software. In financial systems, for example, undetected bugs can lead to significant losses. In healthcare applications, code security and accuracy are essential to maintain patient data integrity. Similarly, startups and tech companies benefit from quicker release cycles and improved codebase maintainability through automated reviews.

1.1.3 Benefits of Using Gemini AI for Code Review:

- Intelligent Analysis: Learns from a wide range of codebases and best practices.
- Bug Detection: Identifies logical and syntactic errors efficiently.
- Security Insights: Flags insecure code that may be vulnerable to attacks.
- Performance Optimization: Suggests refactoring and optimization techniques.
- Scalability: Can handle multiple concurrent code submissions.

1.2 Problem Statement and Objectives:

Current code review processes are inefficient, inconsistent, and prone to error. Static analysis tools offer limited contextual understanding, and human reviewers can be biased or overlook critical issues. There is a need for a robust, intelligent system to automate and enhance code review.

Objectives:

- To develop a web-based application for reviewing source code.
- To integrate Gemini AI for intelligent, real-time code analysis.
- To detect bugs, enforce coding standards, and suggest improvements.
- To support a user-friendly interface with GitHub integration.
- To ensure scalability and security for real-world usage.

1.3 Significance and Motivation for the Project:

With the increasing complexity of modern applications, there is a pressing need for tools that not only assist developers in writing better code but also maintain code integrity and security. This project addresses this need by introducing an AI-powered platform that automates the code review process. The motivation behind this project lies in improving developer productivity, reducing the turnaround time in code review cycles, and increasing software reliability. The project also serves as a step toward building intelligent programming assistants capable of collaborative development with humans.

LITERATURE SURVEY

LITERATURE SURVEY

S. No	Title	Authors / Source	Year	Key Contribution
1	Automated Code Review using Deep Learning Techniques	R. Sharma, P. M. Patil, and S. Deshmukh – IEEE	2020	Proposed a deep learning model using LSTM/CNN to detect code smells and generate automated reviews.
2	A Survey of Machine Learning for Big Code and Naturalness	Miltiadis Allamanis, Earl T. Barr, Premkumar Devanbu, Charles Sutton – ACM Computing Surveys (APA)	2018	Comprehensive overview of ML applications to source code; supports use of language models for coding.
3	An Empirical Study on Learning Bug-Fixing Patches in the Wild via Neural Machine Translation	P. Tufano, A. Watson, G. Bavota, M. Di Penta, D. Poshyvanyk – IEEE	2019	Demonstrated how NMT can generate accurate bug-fixing patches from real-world code.

2.1 Existing Systems

A. Traditional Code Review Methods:

Code reviews are a critical part of modern software development aimed at improving code quality and ensuring maintainability. Traditionally, this has been performed manually via:

Peer Reviews using Pull/Merge Requests:

Platforms like GitHub and GitLab allow developers to submit code changes, which are then reviewed by team members.

Drawbacks:

- Time-consuming.
- Highly subjective.
- Requires domain knowledge and experienced reviewers.

Static Code Analysis Tools:

Tools such as SonarQube, ESLint, and Checkstyle analyze source code for coding standards and potential bugs.

Limitations:

- Limited to syntactic analysis.
- High rate of false positives.
- Lack semantic and contextual understanding.

B. AI-Based Code Review Tools:

AI-driven tools have emerged in recent years to automate the code review process and assist developers in writing better code.

- GitHub Copilot (OpenAI, 2021):

- A code suggestion tool trained on vast open-source datasets.

Limitations in Existing Systems

Drawbacks:

- Not always context-aware. May produce insecure or deprecated code patterns.
- DeepCode (2020, by Snyk):
- Uses machine learning to detect bugs and offer suggestions. Trained on millions of commits and pull requests.

Studies from IEEE & Springer:

"AI-Powered Code Review: Enhancing Software Quality" (IEEE, 2022) highlighted machine learning techniques for analyzing pull requests with up to 82% accuracy in detecting software issues.

"Deep Learning for Automated Code Review" (Springer, 2021) explored NLP models for understanding and recommending fixes.

C. GitHub Integration-Based Systems

Some modern tools integrate with GitHub directly to:

- Automatically review pull requests.
- Comment in line with suggestions.
- Track review metrics over time.

2.2 Proposed System Overview (Your Project)

The proposed project leverages Gemini AI and the MERN stack to create a full-fledged AI-Powered Code Reviewer that:

- Accepts code through a user-friendly React interface.
- Uses a Node.js + Express backend to process and route code.
- Sends code to a Python-based Gemini AI microservice.
- Analyzes for bugs, code smells, formatting issues, and vulnerabilities.
- Returns clear, actionable feedback.

Comparison with Reference Project

A similar open-source project on GitHub — — demonstrates a basic MERN stack implementation for AI review. Your system builds on this architecture by:

- Providing Gemini AI integration for more intelligent and flexible analysis.
- Adding security-aware review to detect vulnerabilities.
- Improving user interface to support real-time feedback and review history.

REQUIREMENT ANALYSIS

Requirement Analysis

3.1 Software Requirements

- Programming Languages: JavaScript (Node.js, React.js), Python (AI integration)
- Frameworks/Libraries: Express.js, React.js, TensorFlow/PyTorch, FastAPI/Flask
- Database: MongoDB (NoSQL)
- Development Tools: Visual Studio Code, Postman, Git, GitHub
- AI Libraries: HuggingFace Transformers, Gemini AI SDK/API
- Operating System: Windows 10/11, macOS, or Linux (Ubuntu preferred)
- Version Control: Git and GitHub for collaborative development

3.2 Hardware Requirements

- Processor: Intel i5 (10th Gen or higher) / AMD Ryzen 5 or equivalent
- RAM: Minimum 8GB (16GB recommended)
- Storage: 256GB SSD minimum (512GB preferred)
- Graphics: Integrated GPU (Dedicated GPU optional for AI tasks)
- Display: HD or Full HD screen (for UI testing)
- Internet: Required for GitHub sync, API access, and updates

3.3 Functional Requirements

- User should be able to paste/upload code for review
- System must analyze the code using Gemini AI
- Display feedback (bugs, vulnerabilities, suggestions)
- Show review history to the user
- Enable real-time suggestions and integration with GitHub
- Provide download option for reviewed code or report

3.4 Non-Functional Requirements

- Performance: Response time under 5 seconds per request
- Scalability: Handle multiple users simultaneously
- Usability: Clean UI and intuitive user experience
- Reliability: Accurate and stable feedback output
- Security: Ensure code confidentiality and data protection
- Maintainability: Modular codebase for easy updates

3.5 Environment Setup

1. Install Node.js, MongoDB.
2. Setup backend using Express.js and link to MongoDB
3. Create frontend using React.js and Axios for API calls
4. Integrate Gemini AI with FastAPI on a separate Python service
5. Connect frontend and backend via REST API
6. Use Git and GitHub for version control
7. Use Postman to test API endpoints before frontend integration

SYSTEM ANALYSIS

4.1 METHODOLOGY

The development of the Al-Powered Code Reviewer project follows a modular and scalable approach, leveraging the MERN stack (MongoDB, Express.js, React.js, Node.js) for full-stack development and Gemini Al for intelligent code analysis. The application is designed to offer real-time code review capabilities with accurate, Al-driven insights. The methodology encompasses both software engineering principles and machine learning workflows to ensure a robust, efficient, and user-friendly system.

The system is built with the following stages in mind:

- Frontend: Developed in React.js to provide an intuitive code submission interface.
- Backend: Built using Node.js and Express.js for handling API requests and communicating with the Al service.
- Al Microservice: Implemented using Python and integrated with Gemini Al to process, analyze, and review the submitted code.
- Database Layer: MongoDB is used to persist user data, code submissions, and review history.

4.2 MODULES

The system is divided into several functional modules, each responsible for a specific aspect of the code review workflow:

4.2.1 DATA HANDLING

This module is responsible for receiving and storing user-submitted code snippets or files. It interacts with the frontend and routes the input to the backend service, which stores data in MongoDB and forwards it to the Al service.

- Input: Source code via form or file upload
- Output: Forwarded to Al model and stored in the database
- Tools: Ados (API calls), Express.js (routing), MongoDB

4.2.2 TEXT PREPROCESSING && FEATURE EXTRACTION

Before analysis, the Al module preprocesses the code to extract relevant features such as syntax structure, control flow patterns, and semantics. This ensures that the Al engine can effectively understand and interpret the code.

- Tokenization
- Syntax tree parsing
- Identification of variables, functions, and logic structures

4.2.3 MODEL TRAINING & EVALUATION

Although Gemini AI is pretrained, additional training and evaluation steps can be incorporated to fine-tune the system for specific coding standards or organization-specific practices.

- Pretrained Gemini AI with optional fine-tuning
- Evaluation using real-world datasets (optional)
- Metrics: Accuracy, Precision, Recall

4.2.4 MODEL PREDICTION

This is the core of the AI functionality. It uses the preprocessed input to generate predictions, including:

- Bug detection
- Code quality suggestions
- Security vulnerability warnings
- Performance optimization hints

The output includes suggestions, warnings, and confidence scores.

4.2.5 WEB APPLICATION

The user-facing web application built with React provides:

- Code editor (with syntax highlighting)
- Submission and review buttons
- Display of AI-generated review results
- History and session tracking

4.2.6 USER MANAGEMENT

This module handles user sessions, profile management, and GitHub token integration (optional). It ensures a personalized experience and saves each user's review history.

- Authentication (optional)
- GitHub integration (future enhancement)
- Review tracking

4.2.7 UTILITY & SUPPORT

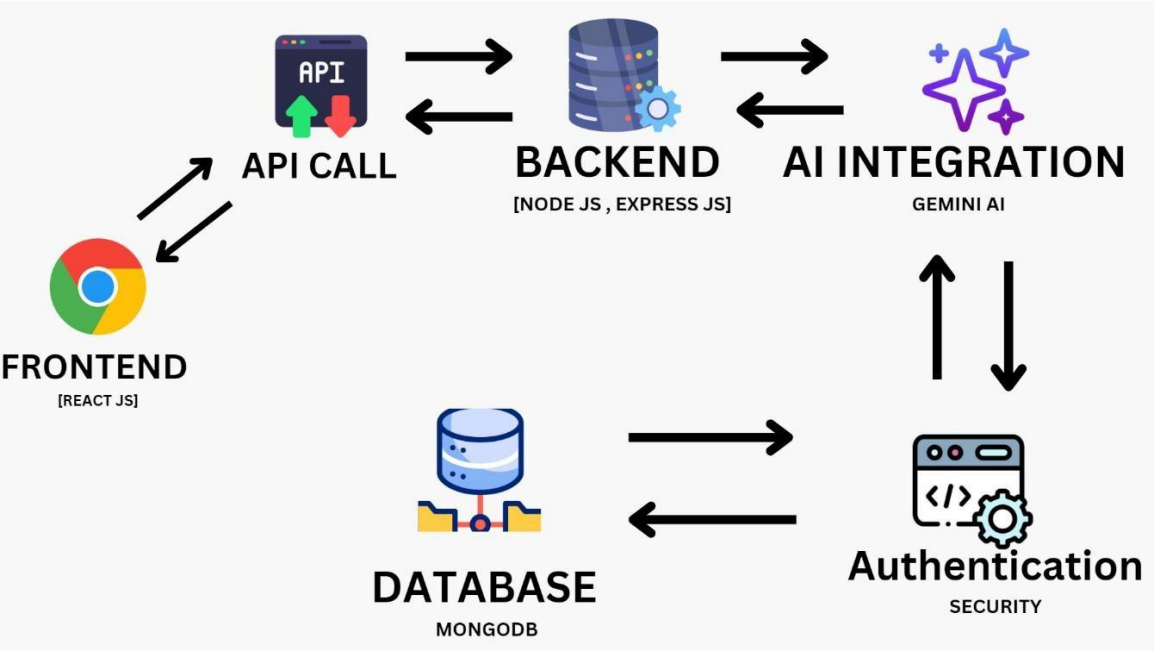
This module includes auxiliary functions that support the system's functionality and reliability:

- API logging
- Error handling
- CORS configuration
- Deployment support for cloud hosting (e.g., Render, Vercel)

SYSTEM DESIGN

System Design

5.1 System Architecture



5.1.1 System Components

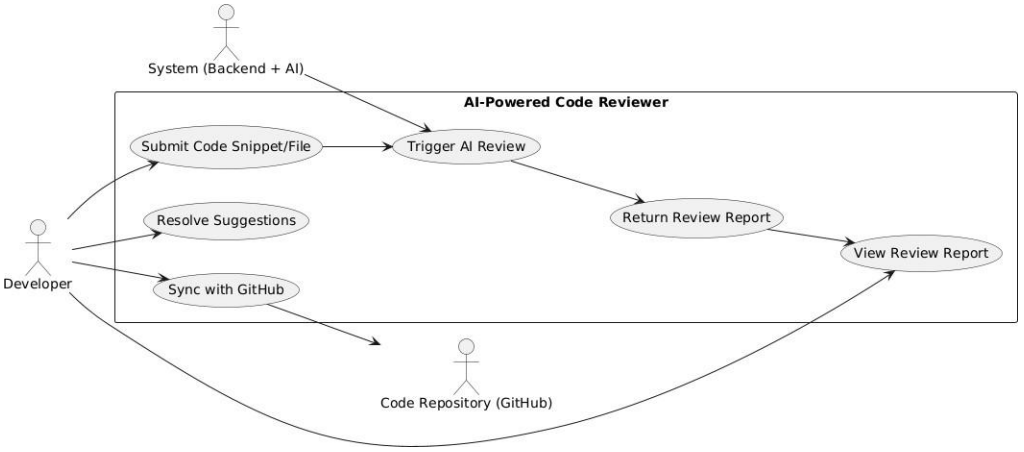
- Frontend: React-based UI
- Backend: Node.js REST API
- AI Service: Flask/Python Gemini AI wrapper for analyzing code
- Database: MongoDB stores:
- Code submissions
- AI review reports
- User profiles & tokens (GitHub token if linked)

System Workflow

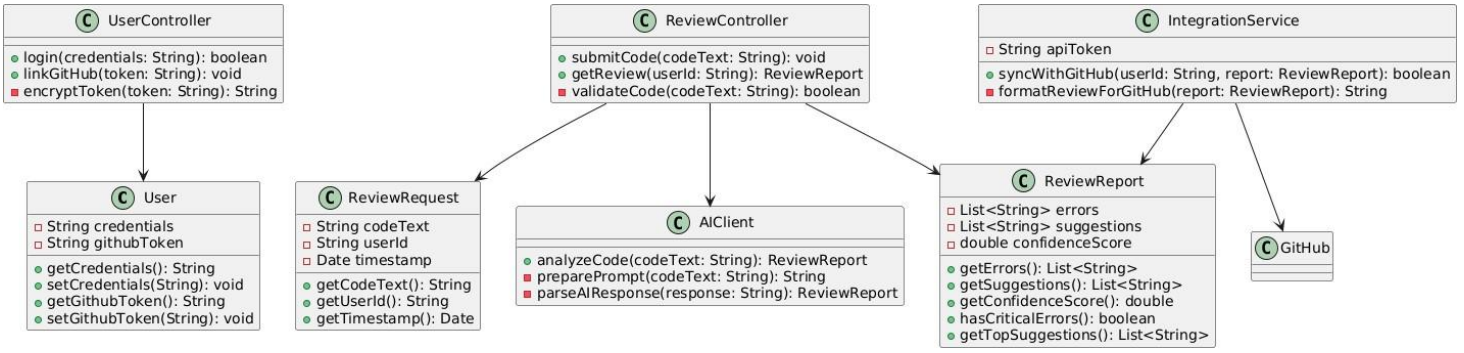
- User submits code snippet/file via UI
- Frontend calls Backend API
- Backend forwards request to Gemini AI microservice
- Gemini AI returns review report (errors, suggestions, confidence score)
- Backend stores review & returns it to frontend
- Frontend displays review to user
- (Optional future) Sync review with GitHub PR/commit

5.2 UML Diagrams

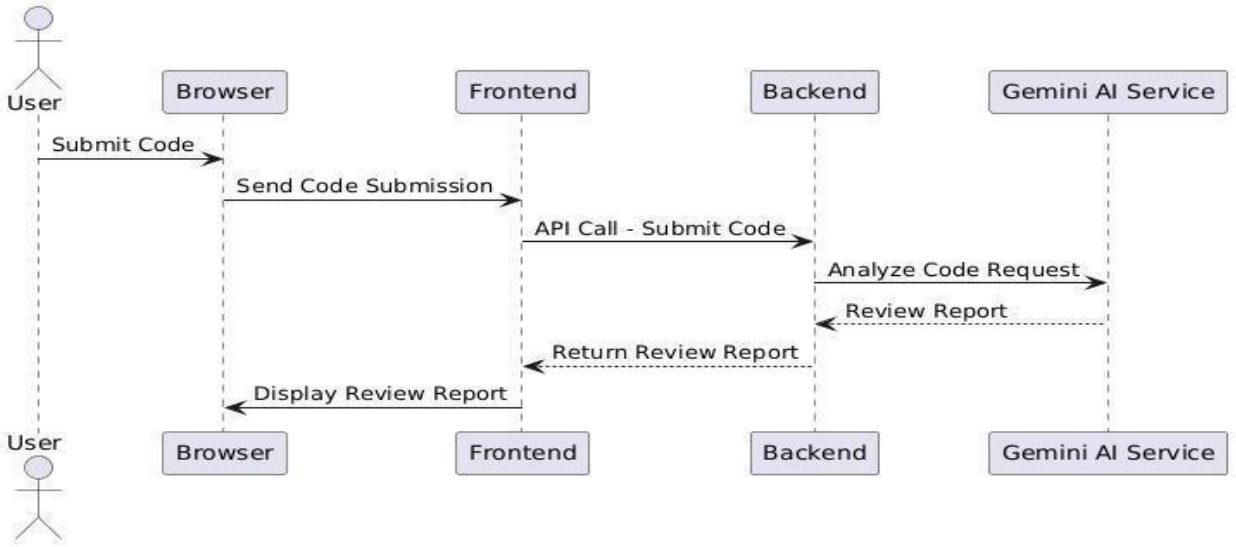
5.2.1 : Use Case Diagram:



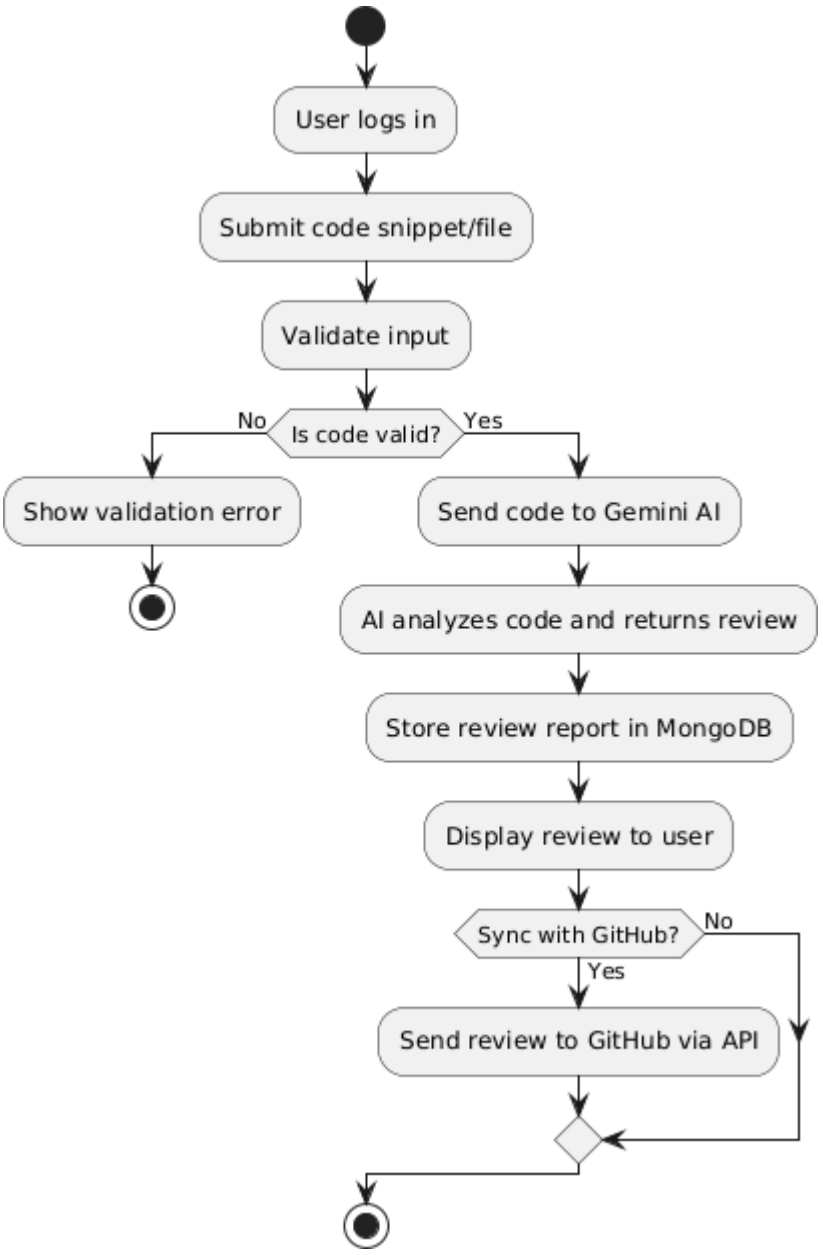
5.2.2 : Class Diagram:



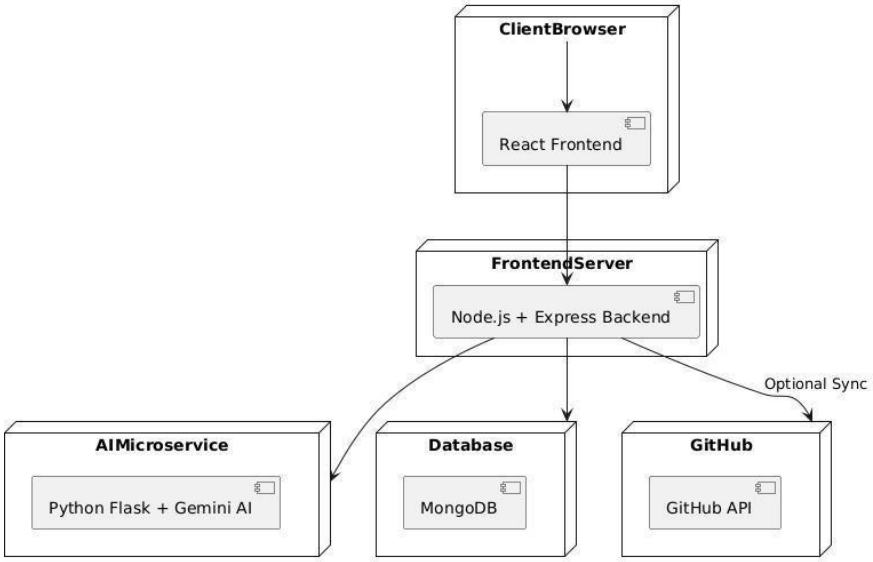
5.2.3 Sequence Diagram:



5.2.4 Activity Diagram:



5.2.5 Deployment Diagram:



IMPLEMENTATION

```
import { useState, useEffect } from 'react'
import "prismjs/themes/prism-tomorrow.css"
import Editor from "react-simple-code-editor"
import prism from "prismjs"
import Markdown from "react-markdown"
import rehypeHighlight from "rehype-highlight"
import "highlight.js/styles/github-dark.css"
import axios from 'axios'
import toast, { Toaster } from 'react-hot-toast'
import './App.css'

function App() {
  const [code, setCode] = useState(` function sum() {
  return 1 + 1
}`)
  const [review, setReview] = useState(``)

  useEffect(() => {
    prism.highlightAll()
  }, [])

  async function reviewCode() {
    const loadingToast = toast.loading("Reviewing your code...")

    try {
      const response = await axios.post('http://localhost:3000/ai/get-review', { code })
      setReview(response.data)
      toast.success("Review completed!", { id: loadingToast })
    } catch (error) {
      toast.error("Failed to review code!", { id: loadingToast })
    }
  }

  return (
    <>
      <Toaster position='top-right' />
      <main>
        <div className="left">
          <div className="code">
            <Editor
              value={code}
              onValueChange={code => setCode(code)}
              highlight={code => prism.highlight(code, prism.languages.javascript, "javascript")}
              padding={10}
              style={{
```

```
                    fontFamily: '"Fira code", "Fira Mono", monospace',
                    fontSize: 16,
                    border: "1px solid #ddd",
                    borderRadius: "5px",
                    height: "100%",
                    width: "100%"
                  }}
                />
              </div>
              <div onClick={reviewCode} className="review">Review</div>
            </div>
            <div className="right">
              <Markdown rehypePlugins={[rehypeHighlight]}>
                {review}
              </Markdown>
            </div>
          </main>
        </>
      )
    }

    export default Appheight: "100%",
    width: "100%"
  }}
/>
</div>
<div
  onClick={reviewCode}
  className="review">Review</div>
</div>
<div className="right">
  <Markdown

  rehypePlugins={[ rehypeHighlight ]}

  >{review}</Markdown>
</div>
</main>
</>
)
}


export default App
```

main.jsx:

```
import { StrictMode } from 'react'
```

```
import { createRoot } from 'react-dom/client'
import './index.css'
import App from './App.jsx'

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <App />
  </StrictMode>,
)
```

Server.js:

```
require('dotenv').config();
const app = require('./src/app');

const PORT = process.env.PORT || 3001;
app.listen(PORT, () => {
  console.log(`Server is running on http://localhost:${PORT}`);
});
```

App.js:

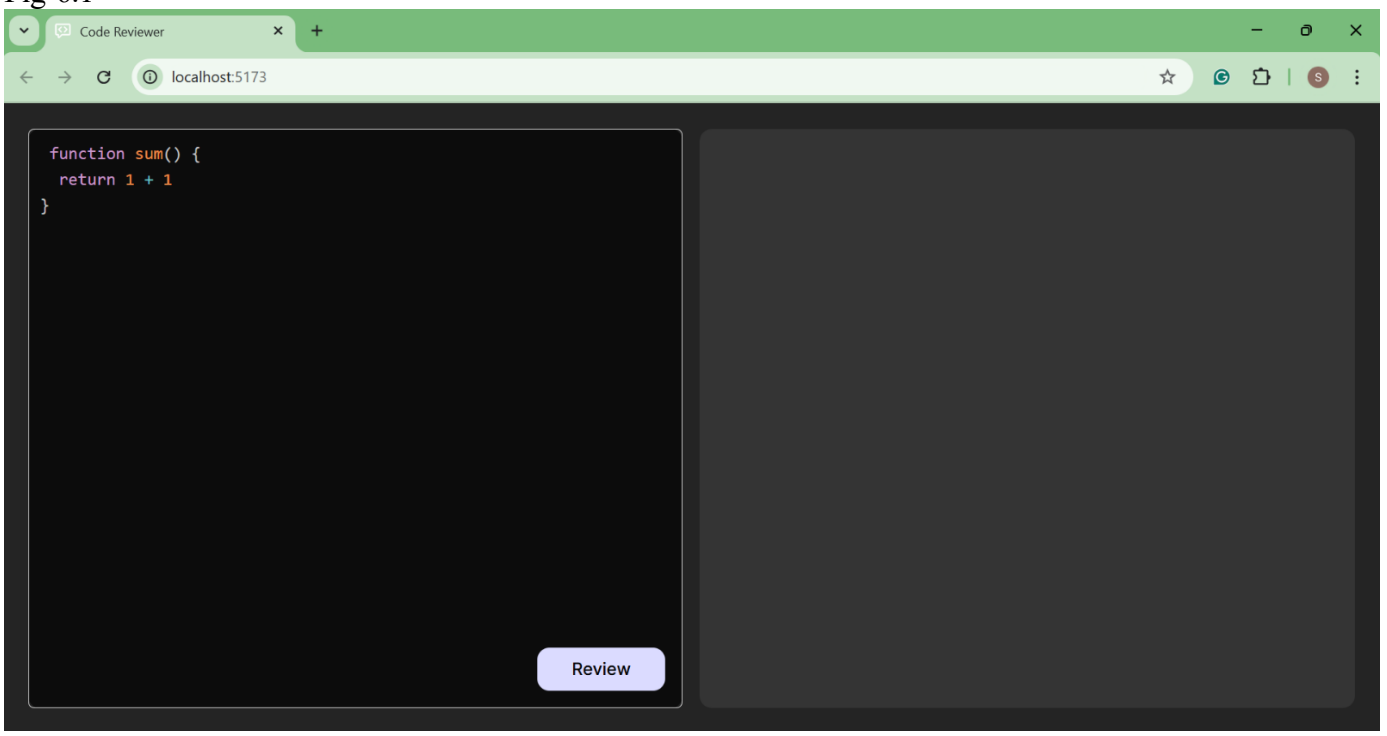
```
const express = require('express');
const mongoose = require('mongoose');
const reviewRoutes = require('./routes/review.routes');
require('dotenv').config({ path: '../.env' });

const app = express();

// Middleware to parse JSON
app.use(express.json());

// Routes
app.use('/api/reviews', reviewRoutes);

// MongoDB connection
mongoose.connect(process.env.MONGO_URI, {
useNewUrlParser: true,
  useUnifiedTopology: true,
}).then(() => console.log('MongoDB connected'))
  .catch((err) => console.error('MongoDB connection error:', err));

module.exports = app;
```

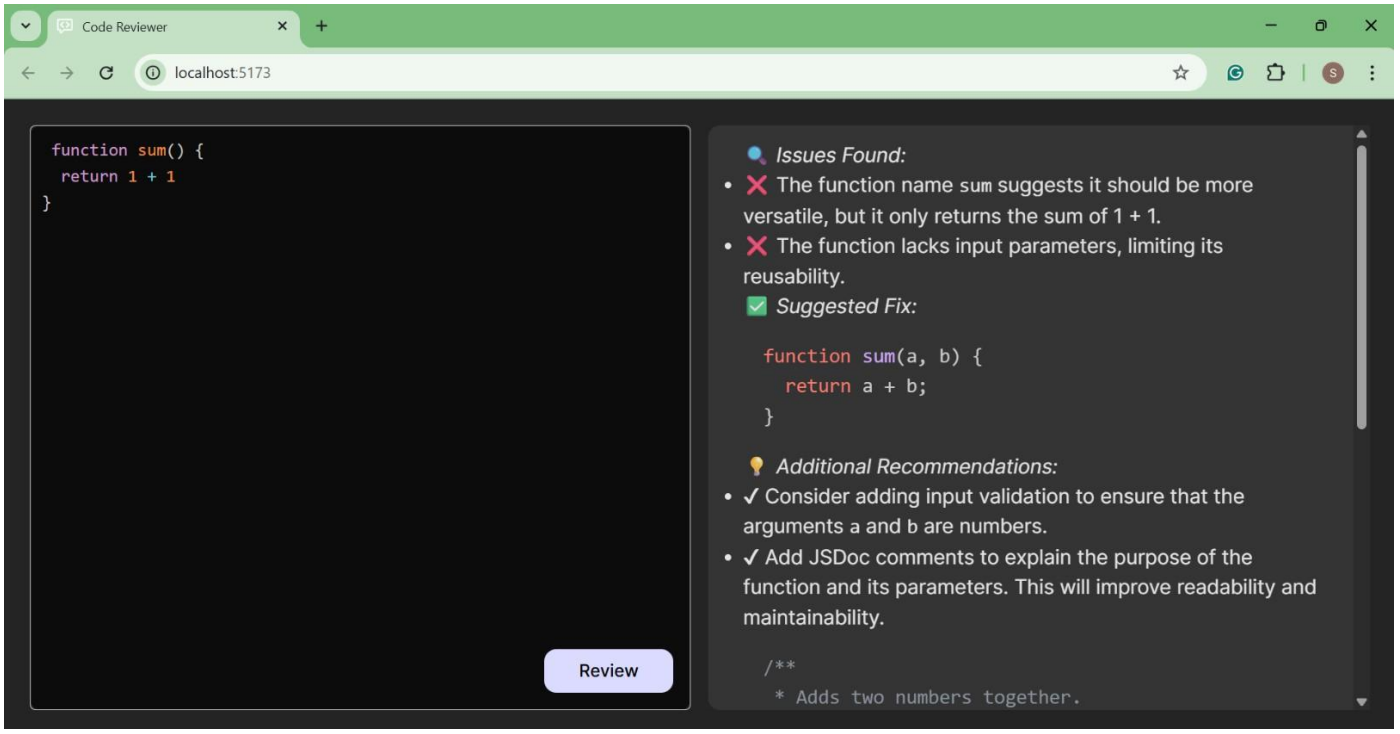
OUTPUT SCREENS

This screen shows the initial state of the Code Reviewer interface. The user interface is divided into two main panes:

- Left Pane: A code editor (likely using a syntax-highlighted component) where users can input source code. In this case, a simple JavaScript function sum() is written that returns 1 + 1.
- Right Pane: Initially empty. This pane is used to display the analysis and review after the "Review" button is clicked.
- Button: The "Review" button triggers the review process, sending the input code to the backend for analysis.

Fig-6.1

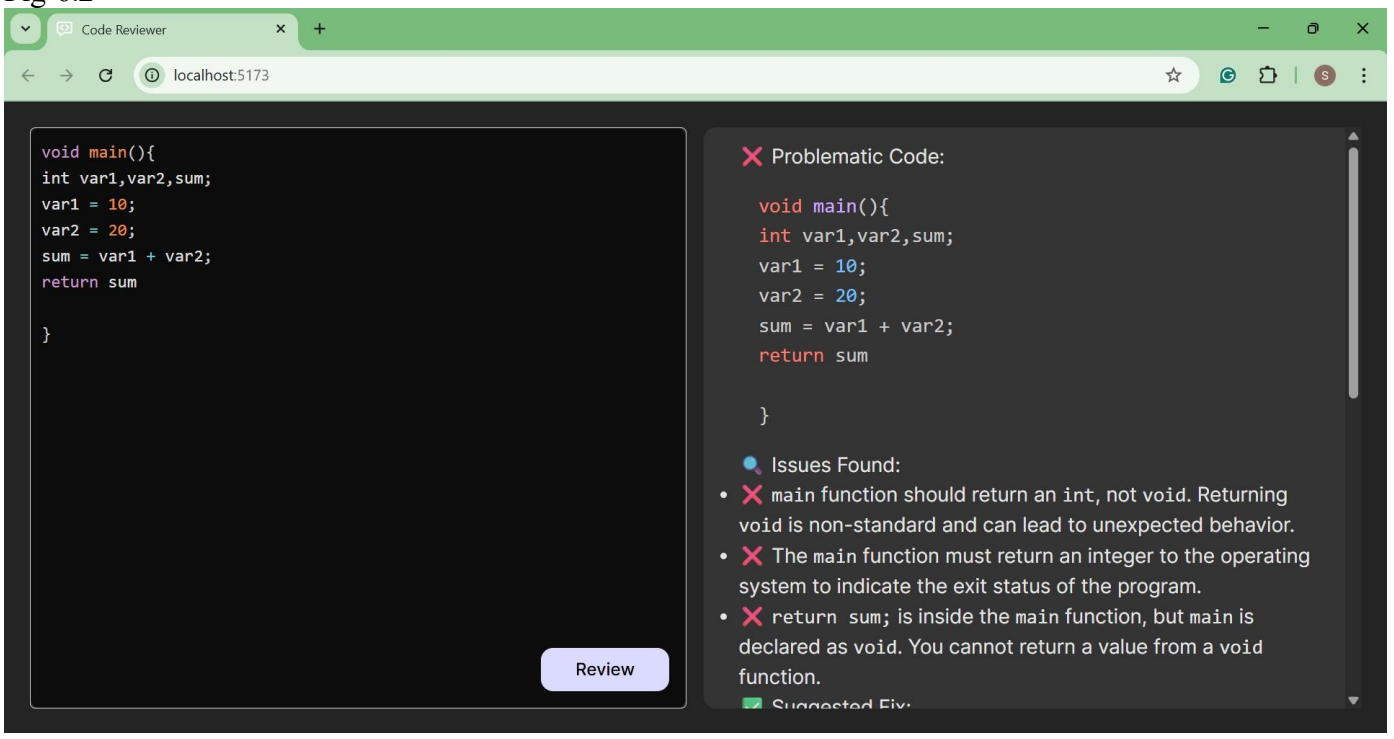


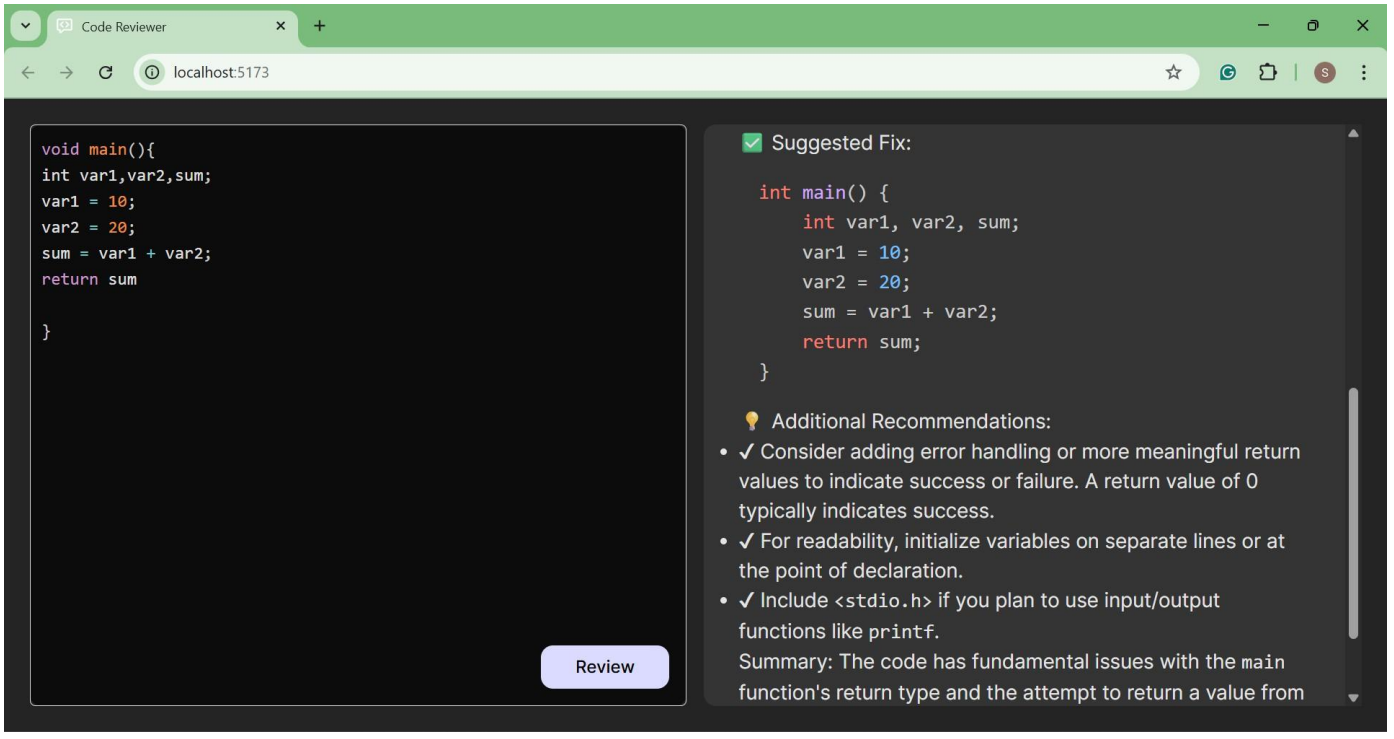


This screen showcases a review for a C program. The code is a simple main() function performing an integer addition, but several issues are identified:

- Problematic Code:
  The function is declared as void main() but contains a return sum; statement.
- Issues Found:
  - The main function should return an int, not void.
  - Returning from a void function leads to undefined behavior.
  - Standard practice requires the main function to return an integer status code to the OS.

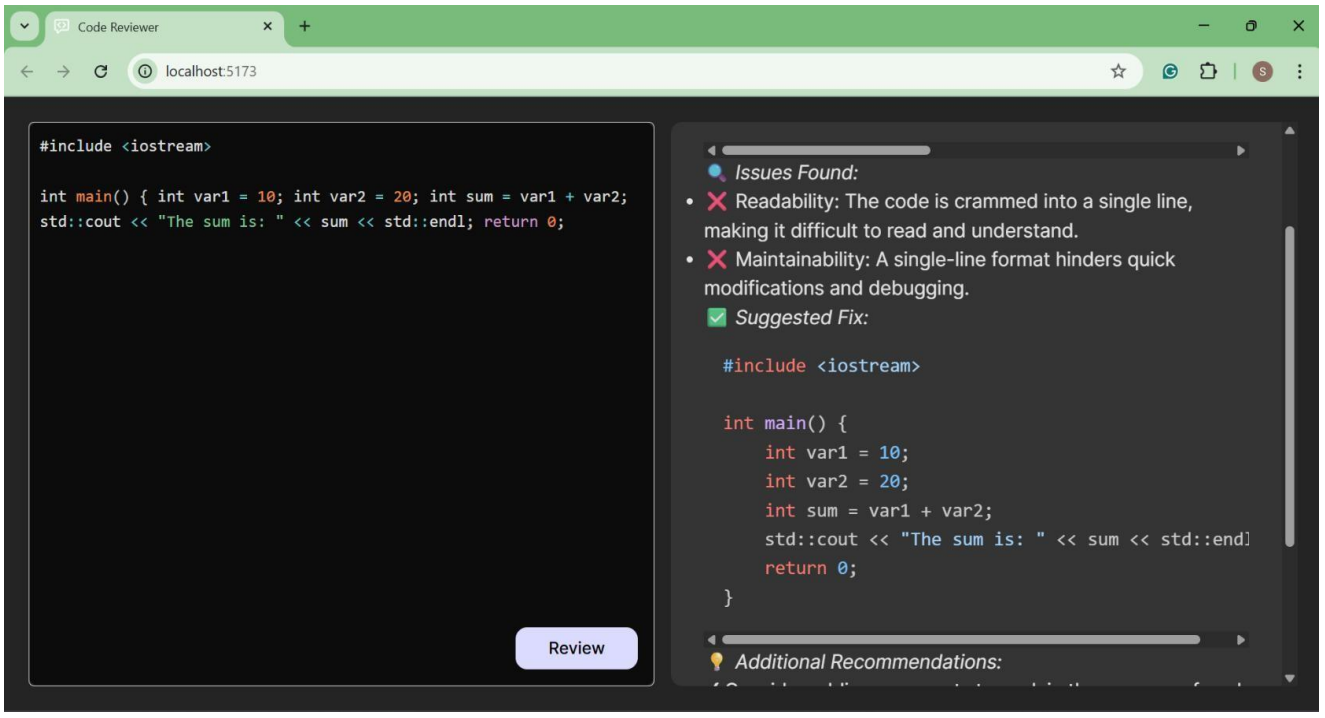
Fig-6.2

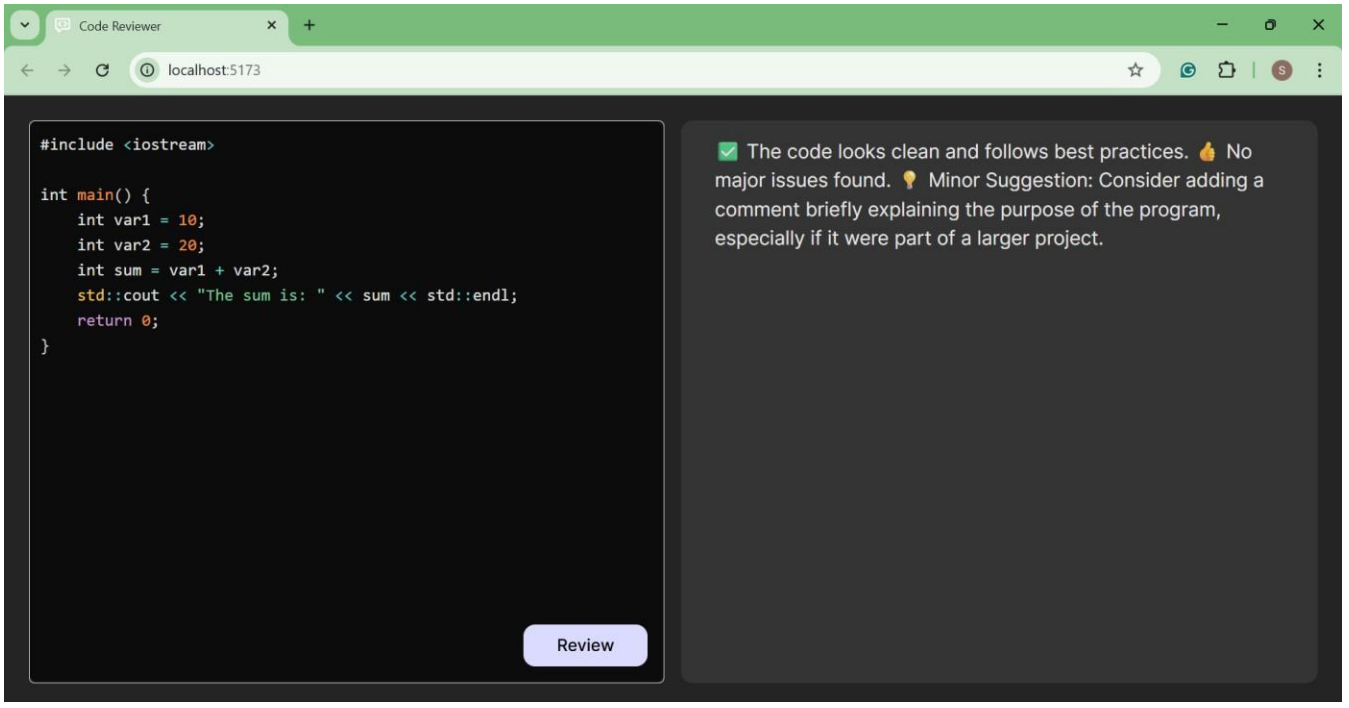




This screen shows the Code Reviewer analyzing a C program's main() function. It identifies and flags critical issues: void main() incorrectly attempts to return a value, causing undefined behavior and violating standard C practice requiring int main() to return an integer.

Fig-6.3





CONCLUSION

Conclusion and Future Scope

9.1 Conclusion

The AI-Powered Code Reviewer project successfully demonstrates the ability to automate code review using advanced AI models (Gemini AI) combined with modern web technologies (MERN stack).

The system allows developers to:

- Submit code snippets or files
- Receive detailed review reports (errors, suggestions, confidence scores)
- Resolve suggestions via UI
- (Optionally) sync reviews with GitHub

Advantages over existing tools:

- More interactive and explainable than static analysis tools (SonarQube, ESLint)
- Does not require code to be public (unlike GitHub Copilot auto-completion)
- Customizable & integrable with any web app
- Provides confidence scores, enhancing transparency

9.2 Future Scope

- To further enhance the project, the following features can be added: Live Linting

- Instant inline linting in the editor itself AI Explainability

- Show WHY each suggestion was made by AI Advanced GitHub Integration

- Full PR comment sync and auto-resolve suggestions in PRs Team Collaboration Features

- Allow team comments on reviews

  Track review progress across team members

- Gamification
  Introduce achievements and badges for clean code contributions

- Mobile UI

  Allow reviewing code from mobile

8.3: REFERENCE

Bavishi, R., et al. (2022). "AI-Powered Code Review: Enhancing Software Quality." IEEE Transactions on Software Engineering.

Zhou, Y., et al. (2021). "Deep Learning for Automated Code Review." Springer.

GitHub Copilot: https://github.com/features/copilot

DeepCode (now part of Snyk): https://snyk.io/product/developer-security/code/

SonarQube: https://www.sonarsource.com/products/sonarqube/

Gemini AI Documentation (Google): https://ai.google.dev/gemini-api

MERN Stack Guide:
MongoDB Official Docs: https://www.mongodb.com/

React Official Docs: https://react.dev/

Express.js Official Docs: https://expressjs.com/

Node.js Official Docs: https://nodejs.org/en/docs

Python Flask: https://flask.palletsprojects.com/